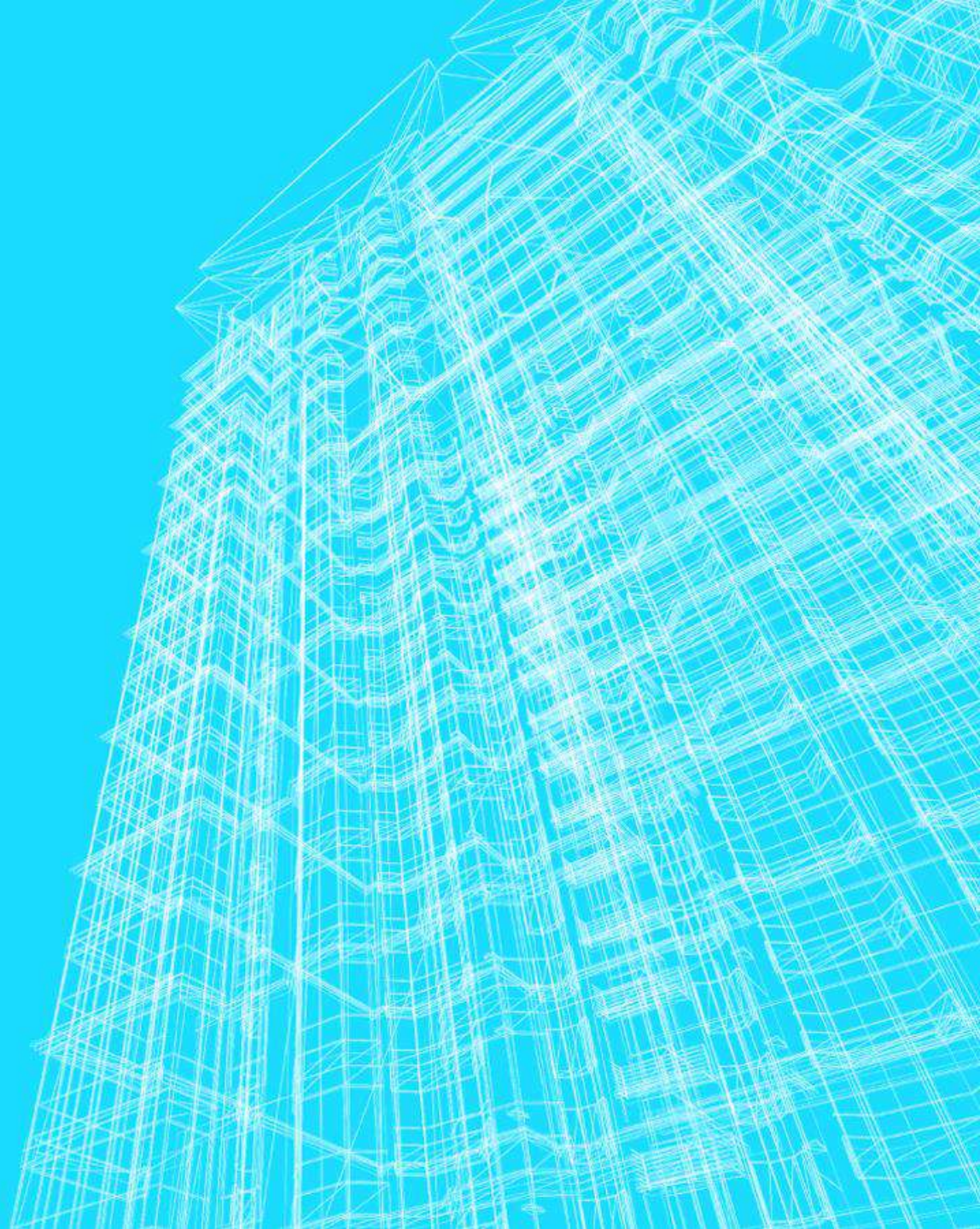
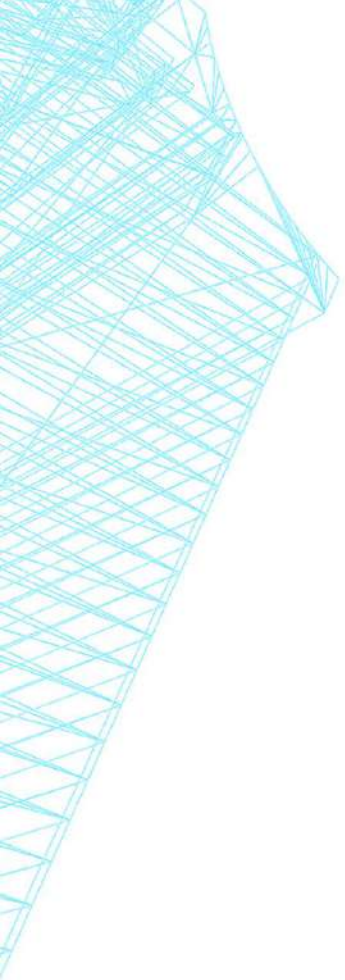


NOTAÇÕES O , Ω E Θ . COMPORTAMENTO ASSINTÓTICO E CLASSES DE COMPORTAMENTO.

João Pedro Oliveira Batisteli





CUSTO DE EXECUÇÃO DE ALGORITMOS

- Utilizamos uma função matemática $f(n)$ para representar o custo estimado de um determinado algoritmo.
- Valores pequenos de n : mesmo os algoritmos ruins custam pouco.
- Logo, a análise de algoritmos é realizada para valores grandes de n .

CUSTO DE EXECUÇÃO DE ALGORITMOS

- Algoritmo com $f(n) = 2^n$.
- Computador que realize 10^9 (1 bilhão) de instruções por segundo.

n	f(n)	Tempo estimado
5	$2^5 = 32$	0,000000032 seg
10	$2^{10} = 1.024$	0,000001024 seg
20	$2^{20} = 1.048.576$	0,001048576 seg
40	$2^{40} = 1.099.511.627.776$	1.099 seg = 18 min
80	$2^{80} = 1.208.925.819.614.629.174.706.176$	$1,2 \times 10^{21}$ seg = 38.308.547 anos

COMPORTAMENTO ASSINTÓTICO

- O comportamento assintótico de $f(n)$ representa o limite do comportamento do custo quando n cresce.
 - Crescimento assintótico da operação mais relevante.
- Em resumo: Comportamento das funções de custo para valores grandes de n .

DETALHES DE NOTAÇÃO

- Inicialmente utilizamos $f(n)$ para denotar uma função de complexidade obtida através da análise de um algoritmo.
- Ex: $f(n) = n^3 + n + \log(n)$.

DETALHES DE NOTAÇÃO

- Agora irá ocorrer uma pequena mudança na notação, a função de complexidade obtida da análise de um algoritmo (ou fornecida) será denotada por $g(n)$.
- $f(n)$ irá denotar uma segunda função de complexidade que queremos descobrir se domina assintoticamente $g(n)$.

NOTAÇÃO O

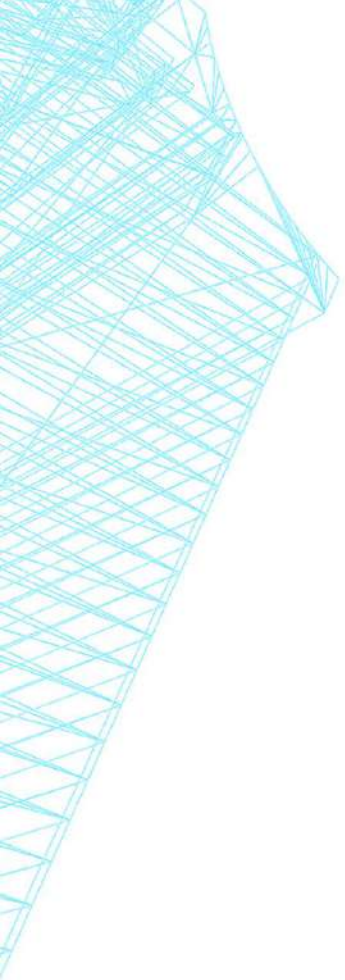
- A função $f(n)$ domina assintoticamente a função $g(n)$ se existem duas constantes positivas c e m tais que, para $n \geq m$, tem-se:
 - $|g(n)| \leq c \times |f(n)|$
- Notação: $g(n) = O(f(n))$

NOTAÇÃO O

- $g(n) = O(f(n))$
 - existe uma constante positiva c , tal que $g(n) \leq c * f(n)$, para algum $n \geq n_0$.
 - n_0 é o ponto de referência no gráfico em $c * f(n)$ passa a ser, sempre, maior que $g(n)$.

NOTAÇÃO O

$$g(n) = O(f(n)), \exists c > 0 \text{ e } m \mid 0 \leq g(n) \leq cf(n), \forall n \geq m$$

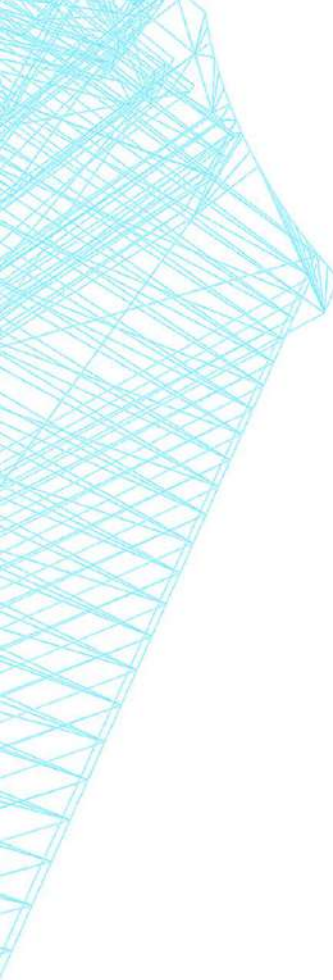


NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$



NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2		
4		
8		
16		
32		
64		

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2	16	32
4	64	88
8	256	272
16	1024	928
32	4096	3392
64	16384	12928

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2	16	32
4	64	88
8	256	272
16	1024	928
32	4096	3392
64	16384	12928

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2	16	32
4	64	88
8	256	272
16	1024	928
32	4096	3392
64	16384	12928

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2	16	32
4	64	88
8	256	272
16	1024	928
32	4096	3392
64	16384	12928

Para $c = 4$ e $n_0 = 16$, $g(n) = O(f(n)) = O(n^2)$

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2	16	32
4	64	88
8	256	272
16	1024	928
32	4096	3392
64	16384	12928

Para $c = 4$ e $n_0 = 16$, $g(n) = O(f(n)) = O(n^2)$

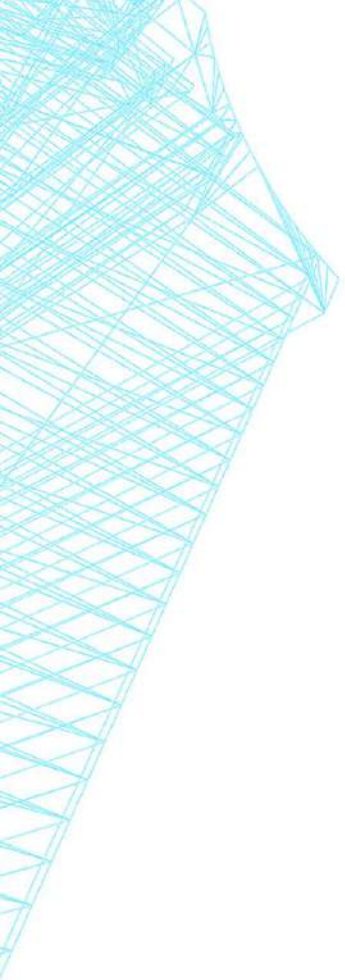
NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = ?$$

$$n^2 + 10n \leq c.n^2$$



DEFINIÇÃO DE LIMITE

“O limite descreve o comportamento de uma função à medida que as entradas ficam cada vez mais próximas de um valor específico, sem necessariamente atingir esse valor.”

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = ?$$

$$n^2 + 10n \leq c.n^2 \div (n^2) \text{ ambos os lados}$$

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = ?$$

$$3n^2 + 10n \leq c \cdot n^2 \div (n^2) \text{ ambos os lados}$$

$$3 + \frac{10}{n} \leq c$$

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = ?$$

$$3n^2 + 10n \leq c \cdot n^2 \div (n^2) \text{ ambos os lados}$$

$$3 + \frac{10}{n} \leq c$$

Quando $n \rightarrow \infty$, $\frac{10}{n}$ tende a 0, mas nunca será 0, logo C pode assumir qualquer valor acima de 4 (pode demorar a encontrar o n_0)

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = ?$$

$$3n^2 + 10n \leq c \cdot n^2 \div (n^2) \text{ ambos os lados}$$

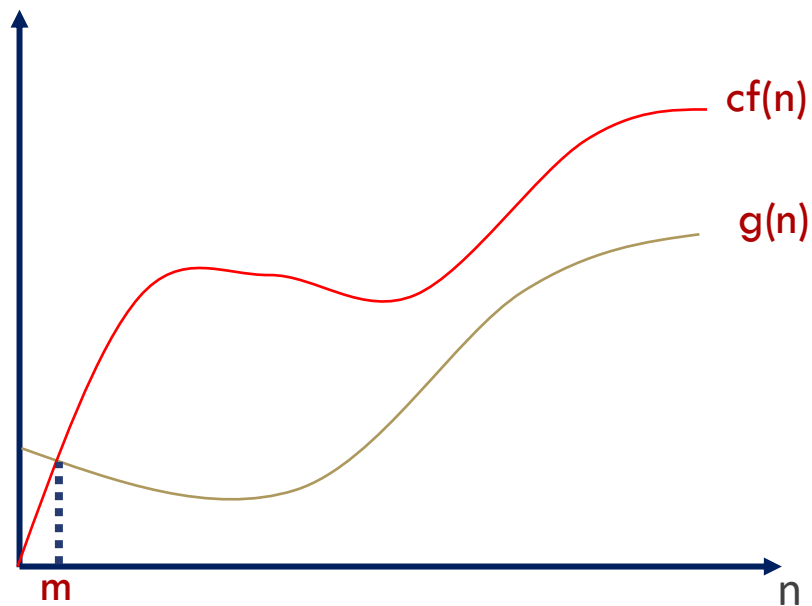
$$3 + \frac{10}{n} \leq c$$

Para facilitar. Para $n_0=1$, teremos que $c \geq 13$ (leva a uma prova mais rápida)

NOTAÇÃO O

$$g(n) = O(f(n))$$

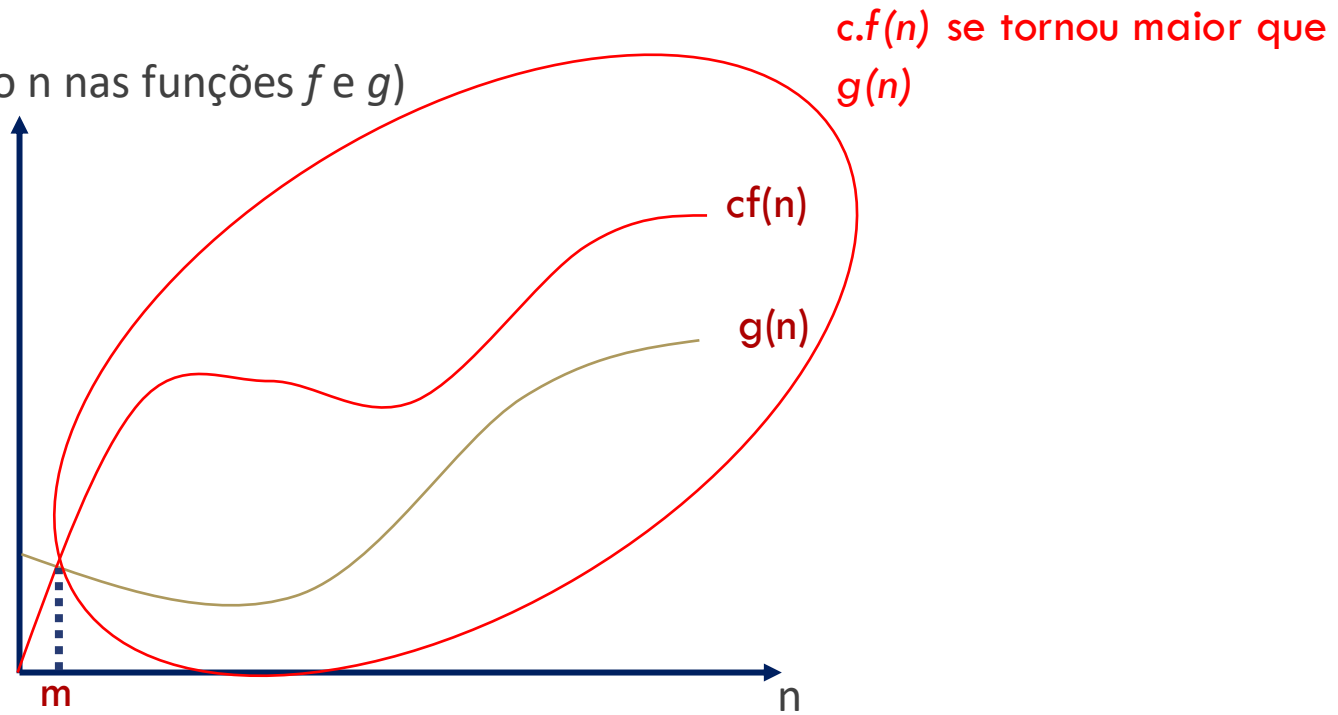
Custo (substituindo n nas funções f e g)



NOTAÇÃO O

$$g(n) = O(f(n))$$

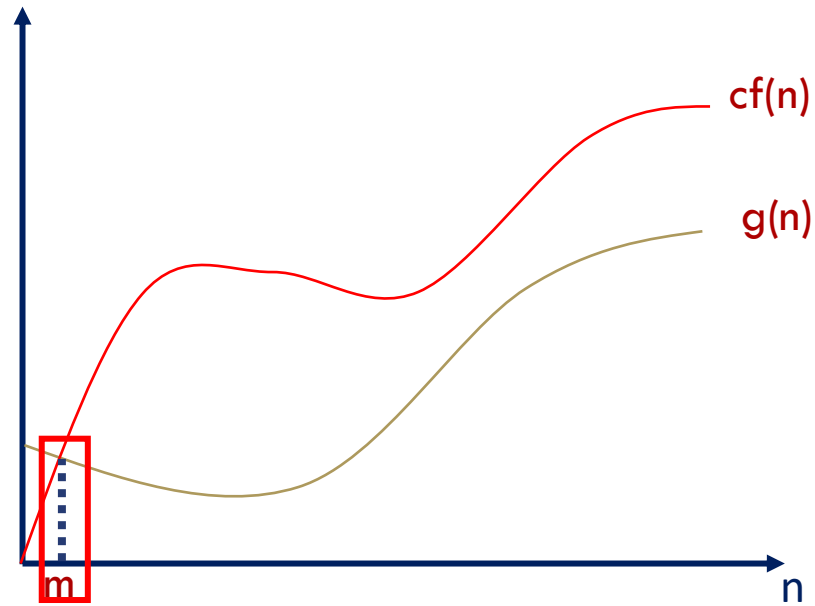
Custo (substituindo n nas funções f e g)



NOTAÇÃO O

$$g(n) = O(f(n))$$

Custo (substituindo n nas funções f e g)



A partir de um certo valor de n (denominado m)

NOTAÇÃO O

- $g(n) = O(f(n))$ implica que $f(n)$ é um limite superior para a taxa de crescimento de $g(n)$.
- Ex: Se o tempo de execução $S(n)$ de um algoritmo é $O(n^2)$, existem constantes c e m tais que, para valores de $n \geq m$, $T(n) \leq c * n^2$.
- $S(n) = O(n^2)$: dizemos que $S(n)$ é da ordem de n^2 .

NOTAÇÃO O

- Se um algoritmo é $O(f(n))$, ele também será $O(g(n))$ para toda função $g(n)$, tal que $g(n) > f(n)$.
- Se um algoritmo é $O(n^2)$ ele também é $O(n^3)$, $O(n^3)$, $O(2^n)$, ...
 - Limite assintoticamente justo.

PROPRIEDADES / OPERAÇÕES

$$f(n) = O(f(n))$$

$$c \times O(f(n)) = O(f(n)) \quad c = \text{constante}$$

$$O(f(n)) + O(f(n)) = O(f(n))$$

$$O(O(f(n))) = O(f(n))$$

$$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

$$O(f(n))O(g(n)) = O(f(n)g(n))$$

$$f(n)O(g(n)) = O(f(n)g(n))$$

PROPRIEDADES / OPERAÇÕES

$$f(n) = O(f(n))$$

$$c \times O(f(n)) = O(f(n)) \quad c = \text{constante}$$

$$O(f(n)) + O(f(n)) = O(f(n))$$

$$O(O(f(n))) = O(f(n))$$

$$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

$$O(f(n))O(g(n)) = O(f(n)g(n))$$

$$f(n)O(g(n)) = O(f(n)g(n))$$

- Algoritmo com três trechos cujos tempos são $O(n)$, $O(n^2)$ e $O(n)$:

- *Qual será a complexidade?*

PROPRIEDADES / OPERAÇÕES

$$f(n) = O(f(n))$$

$$c \times O(f(n)) = O(f(n)) \quad c = \text{constante}$$

$$O(f(n)) + O(f(n)) = O(f(n))$$

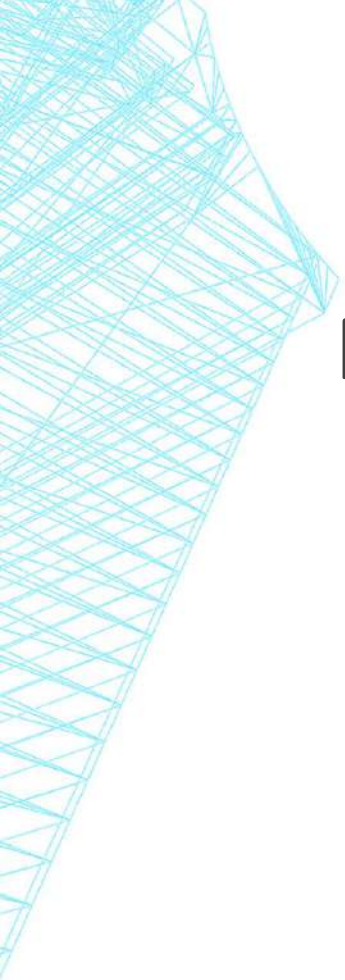
$$O(O(f(n))) = O(f(n))$$

$$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

$$O(f(n))O(g(n)) = O(f(n)g(n))$$

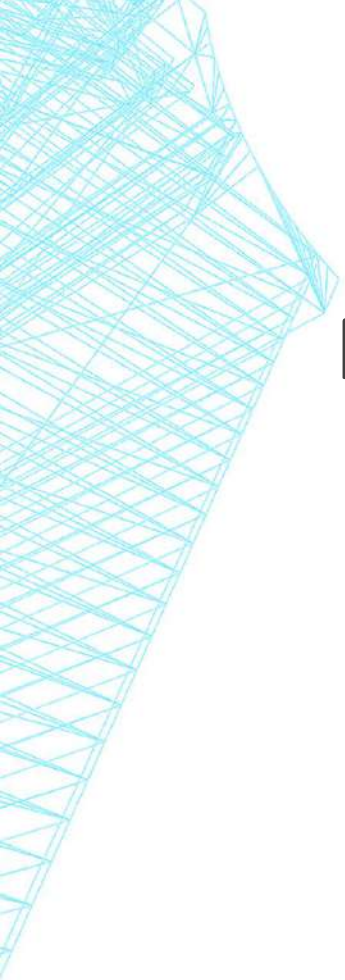
$$f(n)O(g(n)) = O(f(n)g(n))$$

- Algoritmo com três trechos cujos tempos são $O(n)$, $O(n^2)$ e $O(n)$:
 - O algoritmo é $O(n^2)$.



EXERCÍCIO

Prove que $3n^3 + 2n^2 + n + 1 = O(n^3)$



EXERCÍCIO

Prove que $n^{n+1} = O(2^n)$

EXERCÍCIO

Prove que $n^{n+1} = O(2^n)$

$$\frac{n^{n+1}}{2^n} \leq cn^{n+1}$$
$$\leq c 2^n$$

$$\frac{n \cdot n^n}{2^n} \leq c$$

$$(n)^n$$

EXERCÍCIO

Prove que $n^{n+1} = O(2^n)$

$$\frac{n^{n+1}}{2^n} \leq cn^{n+1}$$

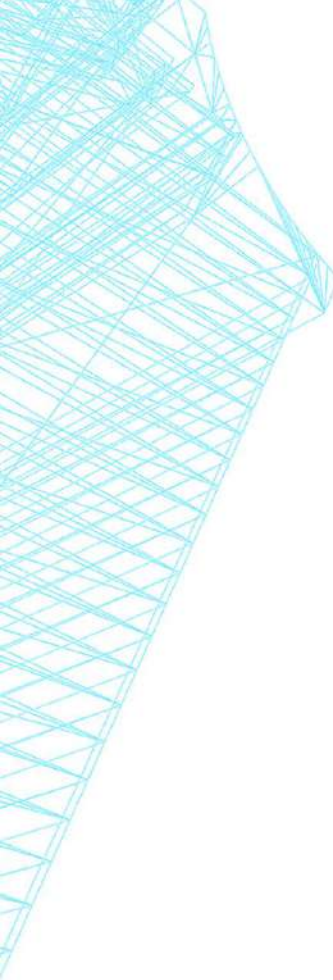
$$\leq c 2^n$$

Vamos analisar o comportamento do lado esquerdo da inequação à medida que n cresce:

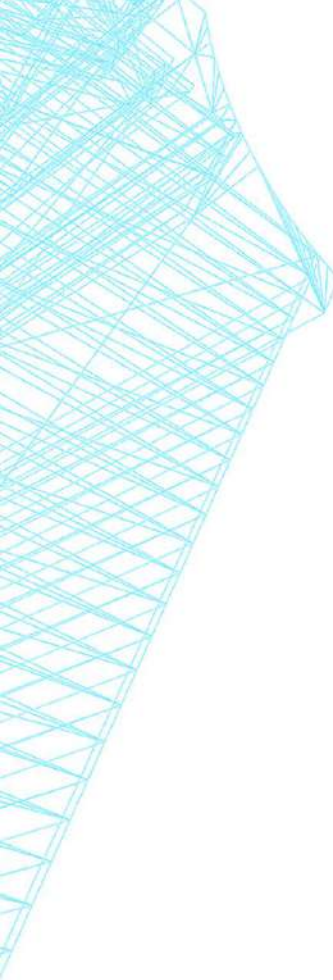
- Para qualquer $n > 2$, a fração $(n/2)$ é estritamente maior que 1.
- Consequentemente, o termo $(n/2)^n$ cresce exponencialmente.
- Multiplicar esse termo por n faz com que a expressão cresça ainda mais rápido.

$$\frac{n \cdot n^n}{2^n} \leq c$$

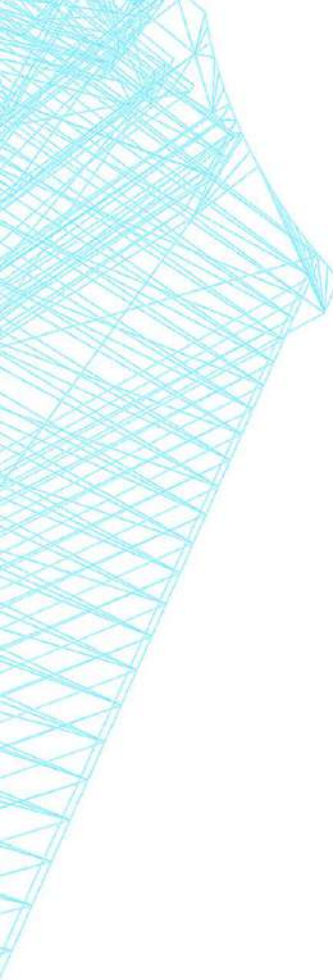
$$\frac{(n)^n}{2^n} \leq c$$



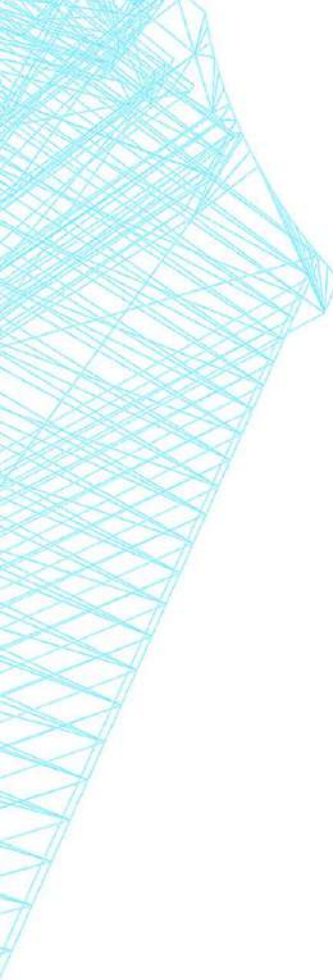
```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor]) ← O(1)
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor]) ← O(1)
7.                 menor = j; ← O(1)
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

} $O(1) + O(1) = O(1)$


```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){ (n-1)+(n-2)+(n-3)...
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

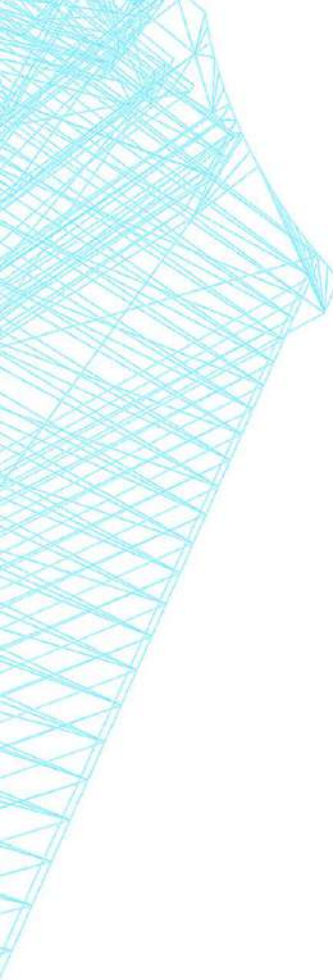
} $O(1) + O(1) = O(1)$

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){ O(n-1)+O(n-2)+O(n-3)...
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

} $O(1) + O(1) = O(1)$

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){ O(n-1)
6.             if (array[j] < array[menor])
7.                 menor = j;           } O(1) + O(1) = O(1)
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

```
1. void algZ(int []array){
2.     int tam = array.length;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){  $O(n-1) = O(n)$ 
6.             if (array[j] < array[menor])
7.                 menor = j;           }  $O(1) + O(1) = O(1)$ 
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```



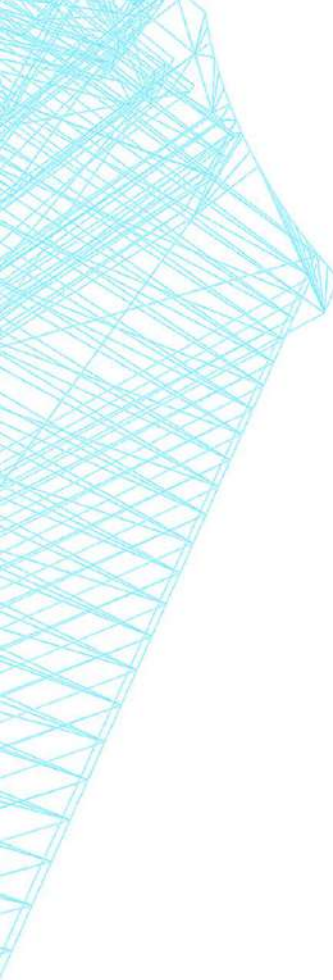
```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){ O(n)
6.             if (array[j] < array[menor])
7.                 menor = j;           } O(1)
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

```
1. void algZ(int []array){
2.     int tam = array.length;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){ O(n)
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(1)$ $O(n)$

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n)$



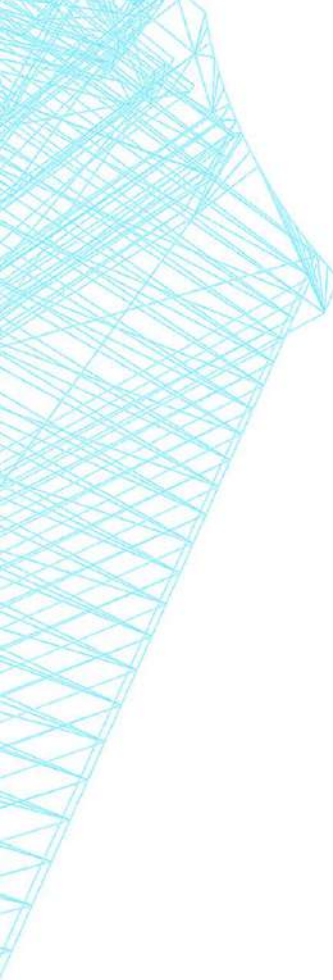
```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n)$

$O(1)$

$O(1)$

$O(1)$



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n)$

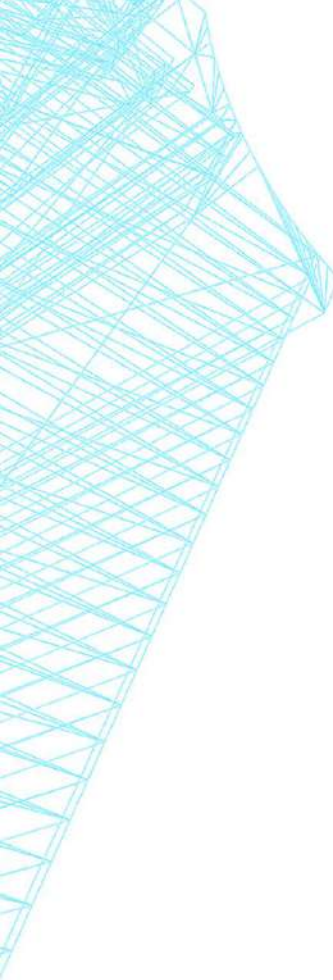
$O(1)$

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n)$

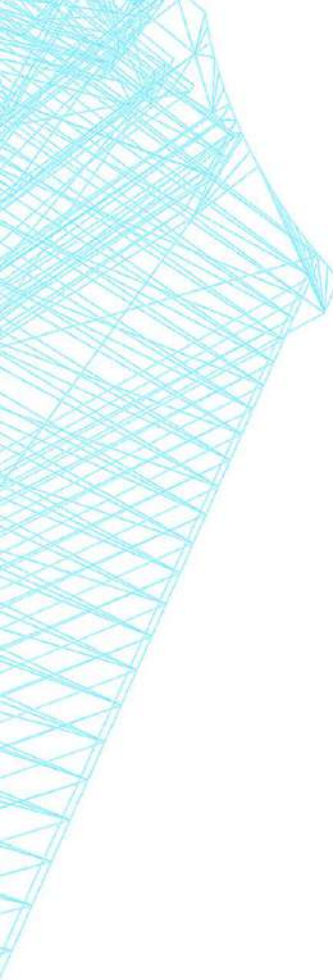
$O(1)$

$O(n)+O(1) = O(n)$



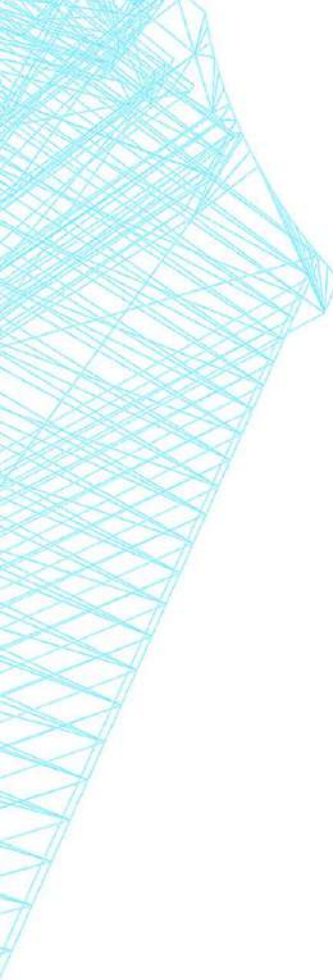
```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n)$



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){ O(n-1) = O(n)
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

} O(n)



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n^2)$